Auto Service AI Triage

Диалоговый помощник для первичного сбора и классификации обращений в автосервис

Назначение проекта

В этом проекте я разработал API-сервис, который помогает последовательно собрать сведения о неисправности автомобиля, сохранить контекст разговора и передать подготовленную заявку в модуль первичного триажа. Система не заменяет мастера и не ставит диагноз: её задача - подготовить структурированное обращение для дальнейшей проверки человеком.

Что умеет система

- Ведёт пошаговый диалог через endpoint POST /agent/chat.
- Запоминает сведения внутри конкретного разговора с помощью LangGraph checkpoint.
- Задаёт по одному короткому уточняющему вопросу, если обязательных данных недостаточно.
- Вызывает собственный tool submit_triage_request после сбора обязательных сведений.
- Классифицирует и сохраняет обращение через существующий сервис триажа.
- Возвращает результат в фиксированной структуре Pydantic.

Основной сценарий

1	Клиент отправляет сообщение в POST /agent/chat.
2	Агент определяет, каких сведений для заявки ещё не хватает.
3	Если данных недостаточно, агент задаёт один короткий вопрос и сохраняет состояние диалога.
4	После получения обязательных сведений агент вызывает submit_triage_request ровно один раз.
5	Существующий триаж классифицирует обращение, сохраняет заявку и возвращает структурированный результат.

Архитектура и работа с данными

Сервис разделён на диалогового агента, инструмент передачи заявки, существующий модуль триажа и два независимых хранилища.



Память и изоляция диалогов

Я разделяю память по сочетанию client_id и thread_id. Приложение создаёт checkpoint ID как SHA-256 от этих двух значений. Поэтому одинаковый thread_id у разных клиентов создаёт разные ветки памяти, а открытые идентификаторы не добавляются в текст сообщения для модели.

- Промежуточное состояние диалога сохраняется в SQLite checkpoint.
- Память относится только к одному thread_id и не является долговременным профилем клиента.
- Готовые обращения хранятся отдельно в SQLite или Turso.

Структурированный ответ агента

Поле	Назначение
message	Ответ клиенту на русском языке.
status	Состояние collecting, submitted или error.
missing_fields	Список сведений, которых пока не хватает.
ticket_id	Номер сохранённой заявки после успешной отправки.
triage	Категория, черновик ответа, confidence и признак escalate.

После успешной отправки status имеет значение submitted, missing_fields остаётся пустым, а ticket_id и triage содержат фактически сохранённый результат.

Безопасность и контрольные факты

Ниже перечислены ограничения и факты, по которым можно проверить, что ответы поискового помощника действительно основаны на этом документе.

Ограничения учебной версии

- Ответы модели и предварительная классификация требуют проверки человеком.
- Агент не ставит диагноз и не сообщает неподтверждённые цены или сроки.
- В демонстрации нельзя использовать реальные персональные данные.
- Отправка сообщений клиенту и другие внешние действия не выполняются.
- Confidence показывает оценку модели, но не гарантирует правильность ответа.

Контрольные факты для поиска

Проверяемый факт	Точное содержание
Обязательные сведения	Автомобиль, описание проблемы, время и обстоятельства появления.
Дополнительные сведения	Необязательны; агент не должен задавать ради них отдельный вопрос.
Изоляция памяти	Checkpoint ID создаётся как SHA-256 от сочетания client_id и thread_id.
Память диалога	SQLite checkpoint хранит состояние только конкретного thread_id.
Журнал заявок	Готовые обращения сохраняются отдельно в SQLite или Turso.
Категории	billing, support, complaint или other.
Проверка проекта	26 автоматических тестов и живой сценарий через ProxyAPI.

Результат реализации

Проект проходит 26 автоматических тестов. Живой сценарий через ProxyAPI подтвердил сбор обращения, однократный вызов собственного tool, восстановление памяти после перезапуска контейнера и изоляцию одинакового thread_id у разных клиентов.